Intelligent Ayurvedic Herbal Plant Identification and Economic Analysis in Sri Lanka

INDIVIDUAL THESIS

H D Sachintha

(IT22168986)

Individual component: Intelligent Ayurvedic Herbal Plant Identification

Group ID: R-26-IT-021

Report submitted in partial fulfilment of the requirements for the

B.Sc. (Hons) Degree in Information Technology

Specialized in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

Submission date to be entered by the author

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. AI assistance used in preparing this thesis is described in Appendix B. I remain responsible for the accuracy, originality, citations, code, data, analyses and conclusions presented. This disclosure does not constitute confirmation of institutional approval for the use of AI.

Student name: H D Sachintha

Student ID: IT22168986

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Mr. Nelum Amarasena		
Co-supervisor: Ms. Kaushika Kahatapitiya		

ABSTRACT

This thesis examines an image-based identification component for the Intelligent Ayurvedic Herbal Plant Identification and Economic Analysis project in Sri Lanka. The component connects mobile image capture to organ-specific classifiers and educational plant information. The evidence comprises the author's proposal, a pinned implementation branch and saved model and evaluation artifacts. The implemented workflow uses a Flutter client, Flask endpoints, EfficientNetV2 classifiers, image-quality heuristics and averaged crop predictions. A YOLO organ detector supports live camera routing, and a separate backend mechanism supports expert review. The active classifiers contain six leaf classes, ten seed or spice classes and eight flower classes. A separate 25-class leaf model exists but is not selected by the reviewed prediction service. Saved validation reports record 14 correct predictions out of 15 for the active leaf model, 59 out of 59 for seed or spice, and 32 out of 35 for the active flower model. These are validation records, not independently reproduced field-test results. Current dataset checks identified exact duplicates in leaf and flower collections, while specimen-level split records and complete source permissions were unavailable. The component demonstrates a concrete identification and information workflow, but its generalization, confidence calibration and stakeholder usability require further assessment. The thesis provides an independent evaluation plan and documents limitations without treating identification as evidence of medical safety or therapeutic effectiveness.

Keywords: Ayurvedic plants, image classification, EfficientNetV2, organ detection, mobile application

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, Mr. Nelum Amarasena, and my co-supervisor, Ms. Kaushika Kahatapitiya, for their guidance and advice during the development of my research proposal. I also thank the Department of Information Technology at the Sri Lanka Institute of Information Technology for supporting the project and providing the academic setting in which this work was developed.

I would also like to thank my fellow members of research project R-26-IT-021 for their contributions to the overall system. Their work on herbal knowledge assistance, cultivation recommendations and plant quality assessment provided the wider context for my Intelligent Ayurvedic Herbal Plant Identification component. I appreciate the shared commitment to bringing these components together into a system that can make plant identification and related information more accessible to users.

Finally, I acknowledge the researchers, dataset contributors and open-source software communities whose work supported the technical foundation of this component. Published studies on plant recognition and transfer learning informed the approach, while the available development tools supported model development and mobile application integration. Their contributions provided valuable resources for this undergraduate research and helped me understand both the possibilities and limitations of image-based plant identification.

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 Overall project context 1

1.2 Individual component context 1

1.3 Problem statement 2

1.4 Research gap 2

1.5 Aim and objectives 3

1.6 Research questions or hypotheses 3

1.7 Scope limitations and assumptions 4

1.8 Individual contribution 4

1.9 Thesis structure 4

2 LITERATURE REVIEW AND RELATED WORK 6

2.1 Review method and scope 6

2.2 Domain and theoretical foundations 6

2.2.1 Convolutional Networks and the Vision Transformer 6

2.2.2 Prediction Error and the Case for Grounded Information 6

2.2.3 Learned Features Class Labels and Information Stores 7

2.2.4 Local Names Taxonomy and Language Accessibility 7

2.2.5 Evaluating Classification and Detection Systems 8

2.3 Related research 8

2.3.1 Knowledge Systems for Sri Lankan Medicinal Plants 8

2.3.2 Image Models in Educational Identification 8

2.3.3 Dataset Construction and Transfer Learning 9

2.4 Existing systems products and techniques 9

2.5 Gap synthesis and conceptual framework 10

3 RESEARCH METHODOLOGY 11
3.1 Research design 11
3.2 Data participants or study materials 11
3.2.1 Classification Datasets and Plant Information 11
3.2.2 Evaluation Sets 12
3.2.3 Participants 12
3.3 Experimental or study procedure 12
3.4 Measures and evaluation metrics 13
3.5 Data analysis 14
3.6 Ethics privacy security and responsible AI 14
3.7 Reproducibility and research artefacts 15
4 PRODUCT AND SOLUTION DESIGN 17
4.1 Requirements 17
4.2 Component architecture within the complete solution 18
4.3 Design alternatives and decisions 19
4.4 Detailed design 19
4.4.1 Dataset Labels and Metadata Schema 19
4.4.2 Dataset Preparation and Training Pipeline 20
4.4.3 Prediction Pipeline and Confidence Warning 20
4.4.4 Organ and Category Resolution 21
4.4.5 Result Presentation and Safety Wording 22
4.4.6 API and Client Design 22
4.5 Standards and professional practice 23
5 IMPLEMENTATION INTEGRATION AND TESTING 24
5.1 Development environment and technology stack 24
5.2 Component implementation 24

5.2.1 Dataset Loading and Model Export 24
5.2.2 Preprocessing Aggregation and Category Resolution 25
5.2.3 Plant Information and Human Review 25
5.2.4 API and Client 26
5.3 Integration with the other components 26
5.4 Verification and software testing 27
5.5 Deployment and operation 28
6 RESULTS AND VALIDATION 30
6.1 Research results 30
6.2 Product and technical results 32
6.3 User or stakeholder validation 33
6.4 Integration results 33
6.5 Summary of findings 34
7 DISCUSSION 35
7.1 Interpretation of findings 35
7.2 Comparison with literature baselines and existing solutions 35
7.3 Research contribution 35
7.4 Product contribution 36
7.5 Limitations and threats to validity 36
7.6 Implications 37
8 CONCLUSION AND FUTURE DIRECTIONS 38
8.1 Conclusion 38
8.2 Achievement of objectives 38
8.3 Individual research and product contributions 39
8.4 Future directions 39
REFERENCES 41

GLOSSARY 43
APPENDICES 44
Appendix A Research instruments or extended results 44
A1 Supported labels and current class distribution 44
A2 Implementation evidence register 46
A3 Independent evaluation instrument 47
A4 Outstanding evidence and validation requirements 48
Appendix B Academic integrity and AI use disclosure 49

LIST OF FIGURES

Figure 3.1 Classifier development and evaluation workflow 13

Figure 4.1 Identification component within the shared application..... 18

Figure 4.2 Upload preprocessing and prediction workflow..... 21

Figure 6.1 Recorded active leaf validation confusion matrix 31

Figure 6.2 Recorded active flower validation confusion matrix..... 32

LIST OF TABLES

Table 1.1 Component objectives and evidence targets 3

Table 2.1 Related work and relevance 9

Table 3.1 Training settings and artifact evidence 15

Table 4.1 Identification requirements and implementation status 17

Table 4.2 Identification relevant API contracts 22

Table 5.1 Verification scenarios and evidence status 27

Table 6.1 Recorded classifier validation results 30

Table 8.1 Objective achievement and remaining evidence 38

Table A.1 Current image class distributions 44

Table A.2 Implementation evidence register 46

LIST OF ABBREVIATIONS

AI	Artificial intelligence
API	Application programming interface
AR	Augmented reality
CNN	Convolutional neural network
CORS	Cross origin resource sharing
EXIF	Exchangeable image file format
F1	Harmonic mean of precision and recall
IoU	Intersection over union
JSON	JavaScript Object Notation
ML	Machine learning
RAG	Retrieval augmented generation
RGB	Red green blue
SHA256	Secure Hash Algorithm 256 bit
TFLite	TensorFlow Lite
UUID	Universally unique identifier
YOLO	You Only Look Once
mAP	Mean average precision

1 INTRODUCTION

1.1 Overall project context

The project, Intelligent Ayurvedic Herbal Plant Identification and Economic Analysis in Sri Lanka, brings together image identification, herbal knowledge assistance, cultivation recommendations and quality assessment. The individual component examined here is Intelligent Ayurvedic Herbal Plant Identification. Its purpose is to turn a photograph into a candidate plant identification and a readable result, while retaining uncertainty and access to educational information. The March 2026 proposal identifies smartphone capture of leaves or flowers as the intended starting point [1].

The implemented branch contains a Flutter client and a Flask backend shared by the project team [2]. This thesis concentrates on the identification service rather than attributing all functions in that shared repository to the individual researcher. Quality grading, Grad-CAM, conversational RAG and economic recommendations belong to neighbouring components. Their presence explains the integration environment but does not establish an individual contribution to those features.

1.2 Individual component context

The individual component, Intelligent Ayurvedic Herbal Plant Identification, focuses on helping users identify Ayurvedic herbal plants through image recognition in a mobile application. It forms part of the broader Intelligent Ayurvedic Herbal Plant Identification and Economic Analysis project and is assigned to H D Sachintha in the project repository [2]. The component connects mobile image capture and upload features with a backend prediction service that processes images and returns a predicted plant label, a confidence score, and related plant information.

The component supports three input categories: leaf or plant images, seed or spice images, and flower images. Each category uses a separate classifier to recognize the labels supported by its model. These categories reflect how users submit images rather than a strict botanical classification; for example, the seed or spice category includes garlic and turmeric. An organ detector also supports live camera input by locating plant parts within a scene. Organ detection and plant identification serve different purposes:

the detector locates a relevant plant part, while the classifier predicts its identity. Together, these functions support image-based identification, although predictions remain limited to the supported classes and require appropriate verification before practical use.

1.3 Problem statement

A person who encounters an unfamiliar plant may have a photograph but no dependable name with which to search for information. A mobile identification workflow can reduce this entry barrier. However, a plausible label can be misleading when the photographed species is outside the model's classes, when an image contains several plants, or when visual details are obscured. The research problem is therefore not simply to output a name, but to connect image classification to understandable uncertainty and an appropriate information display.

The implementation addresses parts of this problem through capture guidance, image-quality warnings, multiple-image averaging, ranked alternatives and a backend expert-review mechanism. These functions cannot by themselves guarantee correctness. The evidence must establish the relationship between training data, deployed artifacts and recorded validation, and must identify where further testing is necessary.

1.4 Research gap

Existing research already demonstrates automated plant recognition and mobile medicinal-herb learning [3], [4]. A broad claim that no such systems exist would be unjustified. The practical gap addressed here is the construction and evaluation of a component within this Sri Lankan project that connects organ-specific classifiers, a mobile capture interface, uncertainty communication and plant-information lookup.

The available evidence does not establish an algorithmic invention or superiority over existing applications. Instead, it supports an implementation contribution and exposes a validation gap: recorded image-level validation is available, but independently collected field-image testing, calibrated confidence and completed stakeholder

assessment are not. Closing that gap is necessary to establish usefulness beyond the repository datasets.

1.5 Aim and objectives

The aim is to develop and assess an image-based identification component that helps users obtain candidate herbal plant labels and educational plant information through a mobile workflow. The following objectives translate the proposal into an evidence-based component scope. Table 1.1 links each objective to an observable deliverable.

Table 1.1 Component objectives and evidence targets

ID	Objective	Evidence target
O1	Identify supported plants from images	Category classifiers and ordered labels
O2	Prepare images and communicate uncertainty	Preprocessing quality record and score warning
O3	Connect identification to educational information	JSON lookup and Flutter result display
O4	Integrate capture and live organ routing	Client contracts and Flask endpoints
O5	Evaluate recognition and supporting functions	Recorded validation and independent evaluation plan

1.6 Research questions or hypotheses

RQ1 asks how organ-specific image classification can be implemented for the supported leaf, seed/spice and flower categories. RQ2 asks what classification performance is recorded for the available artifacts, and what those records establish about generalization. RQ3 asks how uncertainty, input quality and educational information are communicated across the backend and mobile client. RQ4 asks what integration and verification evidence is available and what remains to be tested.

No hypothesis about superiority or a minimum field accuracy is asserted retrospectively. The implementation and its records answer RQ1 and parts of RQ2–RQ4. An independent evaluation plan is provided for the unanswered portions.

1.7 Scope limitations and assumptions

The implementation baseline is branch feature/ayrvedic_plant_recognition at commit [2]. Its active classifiers support six leaf labels, ten seed or spice labels and eight flower labels. The separate 25-class leaf artifact is experimental and is not selected by the prediction service. All deployed-coverage statements and active-model results in this thesis refer to the six-class leaf configuration.

The service assumes that the photographs in a multi-image request concern one intended subject and that the chosen input category is suitable. Optional organ validation examines only the first image. The scope excludes medical diagnosis, authentication of medicinal raw materials, therapeutic effectiveness and suitability for consumption. Botanical label verification, image licensing and the exact original training commands remain unresolved. These constraints affect interpretation rather than being hidden behind a high validation score.

1.8 Individual contribution

The individual component scope is the image-based identification workflow: image preparation, category-specific prediction, organ detection, educational information lookup and the mobile identification interface [2]. The repository assigns this component to H D Sachintha. The shared API, application framework and supporting services are discussed as integration dependencies; their inclusion does not establish sole authorship. File-level ownership and the division of implementation and experimental work are not established by the available contribution records.

The expert-review backend and educational metadata are relevant supporting mechanisms. Their implementation is discussed because it changes identification behaviour, without claiming that an expert study was completed or that every metadata entry was clinically validated.

1.9 Thesis structure

Chapter 2 reviews recognition methods and related systems. Chapter 3 explains the methodology, datasets and evaluation boundaries. Chapter 4 describes requirements and design. Chapter 5 presents implementation, integration and software verification. Chapter 6 reports archived validation results and dataset checks. Chapter 7 discusses

their implications and limitations. Chapter 8 presents conclusions and future work. Appendix A contains class distributions, evidence paths, a proposed evaluation protocol and outstanding evidence requirements; Appendix B discloses AI assistance.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The review used targeted searches for plant identification, medicinal-herb mobile systems, transfer learning, EfficientNetV2, MobileNetV2 and confidence calibration. Bibliographic details were checked against primary papers, publisher records and official technical documentation. This is a focused narrative review, not a systematic review: no exhaustive search count, screening protocol or completeness claim is made.

Related work was selected for its relevance to visual feature learning, mobile interaction, data conditions or uncertainty. The comparison deliberately avoids ranking accuracy values across incompatible datasets. The review focuses on the identification component, while Appendix A provides the implementation evidence and proposed evaluation instruments.

2.2 Domain and theoretical foundations

2.2.1 Convolutional Networks and the Vision Transformer

A convolutional classifier learns local visual patterns and combines them into a representation used for prediction. Transfer learning reuses parameters from a larger source task and adapts a classification head to the target labels. Deep-Plant investigated this approach for 44 plant species using the MalayaKew leaf dataset and explored learned feature representations [3]. Its conditions and taxa differ from the current herbal datasets, so its outcomes are background evidence rather than a baseline result for this project.

Vision Transformers process image patches as a sequence and provide a distinct architecture family [5]. They are relevant as a design alternative, but no transformer classifier is selected by this component. Language-model implementation remains outside this identification component.

2.2.2 Prediction Error and the Case for Grounded Information

A classifier can assign a high score to the wrong class, particularly when its input differs from the training distribution. Guo et al. distinguish confidence calibration from classification accuracy and investigate post-processing calibration methods [6]. The

present service averages softmax outputs and labels the resulting value confidence; no recorded calibration analysis accompanies that value. A 95 percent warning boundary is therefore a software setting, not proof of a 95 percent probability of correctness.

The information display is grounded in JSON lookup records rather than generated by the identification classifier. A correct lookup establishes that a label maps to stored text, not that the text is medically reliable. This separation prevents a successful image classification from being treated as validation of traditional-use claims.

2.2.3 Learned Features Class Labels and Information Stores

EfficientNetV2 is a convolutional architecture family designed around training speed and parameter efficiency [7]. In this component, ImageNet-pretrained B0 or B1 backbones feed a task-specific head. The deployed B0 classifiers and the separate B1 leaf artifact are identifiable from their serialized configurations. Their learned features are internal to the classifier; the implementation does not search a vector store to identify the photographed species.

The ordered labels.json files associate output positions with plant labels. Information storage has a separate purpose: benefits.json and benefits_supplement.json hold educational records, while SQLite holds verification requests. Keeping these roles separate makes label-order checks and metadata coverage checks meaningful.

2.2.4 Local Names Taxonomy and Language Accessibility

The proposal seeks accessible plant information for Sri Lankan users [1]. Local and scientific names can support that aim, but the active label files mix scientific labels, common names and inconsistent spelling. The flower labels include broad names such as Rose and Yellow Daisy. These cannot be assumed to resolve a single botanical species without a documented taxonomy.

The reviewed identification interface uses English labels and explanatory text. The wider project's bilingual assistant does not establish bilingual identification output. Sinhala naming should be added through reviewed metadata and tested for consistent taxon mapping, rather than inferred from the neighbouring conversational component.

2.2.5 Evaluating Classification and Detection Systems

Image classification evaluation compares predicted labels with reference labels. Accuracy describes the overall correct fraction; per-class precision, recall and F1 show how individual classes behave. Macro averages give classes equal weight, while weighted averages reflect support. These measures answer different questions when class counts are unequal.

Object detection also evaluates localization. The organ detector's report therefore includes precision, recall and mean average precision, whereas classifier reports contain classification matrices. Neither score substitutes for the other. A camera workflow can fail through an incorrect organ box even if the downstream classifier performs well on prepared images.

2.3 Related research

2.3.1 Knowledge Systems for Sri Lankan Medicinal Plants

Senevirathne et al. describe a mobile assistive tool for identifying and learning medicinal herbs, with augmented-reality presentation [4]. This establishes relevant local precedent and weakens a claim that mobile medicinal-herb learning is entirely new. The accessible institutional record was sufficient to verify publication identity and high-level scope, but not to reproduce its experiments. No unverified numerical result is copied into the comparison.

The present component is evaluated as a bounded implementation within an integrated project. Its JSON information lookup is simpler than a comprehensive knowledge system. Expert approval of the stored information and a documented botanical vocabulary would be necessary before calling that information authoritative.

2.3.2 Image Models in Educational Identification

Leafsnap combines computer vision with an electronic field-guide setting [8]. Its leaf-recognition approach illustrates why the image acquisition process matters alongside classification. Deep-Plant uses learned CNN features and visualization to address plant discrimination [3]. Together, these studies support the relevance of visual identification but do not establish reliability for this component's photographs.

The practical comparison is between assumptions: prepared leaf images, uncontrolled mobile scenes, restricted classes and broad flora coverage. A model trained for one of these conditions cannot be presumed to solve the others. The thesis therefore places more weight on class scope and split design than on isolated headline accuracy.

2.3.3 Dataset Construction and Transfer Learning

MobileNetV2 uses inverted residuals and linear bottlenecks to build efficient mobile-oriented models [9]. The training script offers it as an alternative to EfficientNetV2, but the active artifacts inspected here contain EfficientNetV2B0. Without a matched experimental comparison, the choice can be explained as an implemented option with a plausible efficiency rationale, not as empirically optimal for Sri Lankan herbs.

Transfer learning makes a small target dataset workable, but does not repair incorrect labels, missing classes or duplicate images. The current evidence supports the need for dataset curation and an independent test collection before architecture comparisons are used to justify stronger claims.

2.4 Existing systems products and techniques

Pl@ntNet provides AI-supported plant identification and community review of observations [10]. It is a useful example of coupling recognition with human participation. The current backend also supports expert requests, but that does not establish an equivalent community or review capacity. Table 2.1 summarizes the evidence-based comparisons without presenting cross-dataset scores as a league table.

Table 2.1 Related work and relevance

Source	Approach and setting	Relevance and comparison limit
[3] Deep Plant	Transferred CNN features; 44 Kew species	Supports learned features; different taxa and acquisition
[4] Medicinal herb mobile tool	Local identification and learning; AR presentation	Prior local system; full experimental protocol not inspected
[8] Leafsnap	Visual leaf identification and electronic field guide	Capture assumptions matter; different application scope

Source	Approach and setting	Relevance and comparison limit
[7] EfficientNetV2	Efficient convolutional architecture family	Actual active backbone; paper does not evaluate this dataset
[9] MobileNetV2	Mobile oriented convolutional architecture	Training option; no matched component comparison
[6] Calibration	Measures score correctness alignment	Supports caution about interpreting confidence
[10] Pl@ntNet	AI identification with community observation review	Human review precedent; no comparable study performed here

2.5 Gap synthesis and conceptual framework

The conceptual framework links input suitability, category selection, classifier inference, confidence communication and educational lookup. The classification output remains a candidate identification. Supporting information is attached only after a label is selected, and human review is an additional verification route rather than an automatic guarantee.

The main unresolved relationship is between performance on the repository validation split and performance on new field photographs. Data provenance, duplicate-aware grouping, taxonomic review and calibration mediate that relationship. These become explicit evaluation requirements in Chapter 3 and future directions in Chapter 8.

3 RESEARCH METHODOLOGY

3.1 Research design

The component uses a development-and-evaluation approach to image-based herbal plant identification. The design connects mobile image acquisition, category selection, image preprocessing, transfer-learning classifiers and educational information retrieval. The March 2026 proposal establishes the intended research scope [1], while the versioned implementation and saved evaluation outputs document the available technical work [2]. The methodology below describes the implemented procedure and identifies where original execution records are incomplete.

Implementation descriptions are supported by source code and model configurations. Numerical recognition results are taken from archived validation reports. Dataset counts, exact-duplicate checks and confusion-matrix arithmetic are secondary verification activities carried out during thesis preparation. No new model-training run, independent field experiment or participant study is reported. The available evidence therefore supports an implementation account and an analysis of recorded validation performance, rather than a completed independent assessment of deployment reliability.

3.2 Data participants or study materials

3.2.1 Classification Datasets and Plant Information

The current dataset tree contains 1,793 leaf images across 25 class folders, 294 seed/spice images across ten folders and 171 flower images across eight folders. Counts include recognized image suffixes and are file counts, not counts of independent specimens. Appendix A provides the class distributions. The older dataset audit records a six-class plant collection with 76 images, whereas the original dataset path is absent from the reviewed tree. It cannot be assumed that this audit exactly reconstructs the data used by the active six-class model.

PROJECT_GUIDE.md identifies the AI4A Lab Kaggle herb classification dataset as a potential source [11]. The available records do not map every image in the leaf, seed or spice, and flower collections to that source. Per-image licences, acquisition dates, specimen identifiers and botanical reviewer records are not documented. Dataset

provenance and permission to redistribute the collections are therefore unverified. Folder names serve as computational class labels and are not evidence of expert botanical identification.

3.2.2 Evaluation Sets

The training script uses a stratified train/validation split, with a default validation fraction of 0.2 and random_state 22168986. It does not create an independent test partition. Archived validation supports are 15 images for the active leaf model, 59 for seed or spice, 35 for the active flower model and 359 for the separate 25-class leaf model. The historical 76-image audit does not identify the exact subset used for the 15-image validation report. Without the original split manifest, the relationship between these records cannot be reconstructed from counts alone.

No immutable split manifest links all these samples to original specimens or captures. A duplicate-aware specimen split is therefore an evaluation improvement, not a documented feature of the completed training procedure.

3.2.3 Participants

The proposal identifies general users, practitioners and other stakeholders [1]. No participant recruitment record, consent form, demographic table, usability score or expert labeling study was located for this identification component. Consequently, no completed participant study is reported. A future study will require a supervisor-approved protocol, task definitions, consent and a clear distinction between botanical reviewers and ordinary interface users.

3.3 Experimental or study procedure

The implemented classifier procedure enumerates sorted class folders and image paths, assigns integer labels, performs the stratified split and creates TensorFlow datasets. Images are decoded to three channels and resized to 224 by 224. Training uses online augmentation, pretrained feature extraction, a learned head and subsequent fine-tuning. The script exports Keras and TFLite artifacts, labels, logs and a classification report.

Figure 3.1 describes this verified procedure and separates the future independent test stage. At inference, the preprocessing path differs from training: uploaded images

receive orientation correction, conservative cropping and mild enhancement before multi-crop prediction. The saved validation routine predicts resized validation batches; it does not execute the complete API preprocessing and eight-view averaging procedure. An endpoint-level evaluation is needed to measure the deployed workflow.

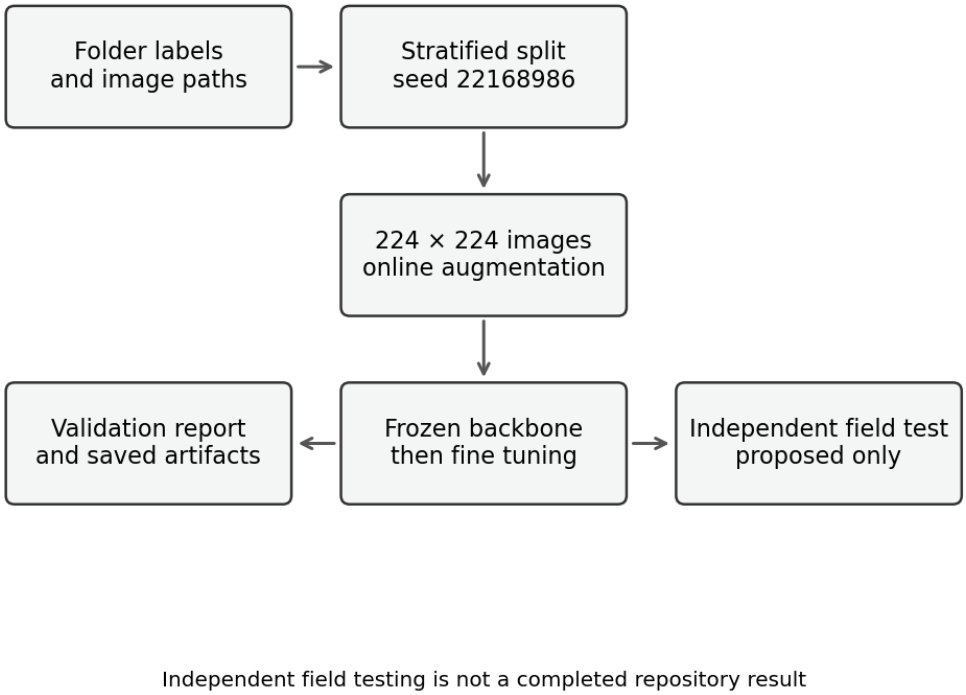


Figure 3.1 Classifier development and evaluation workflow

The preparation-stage audit counted current dataset files, hashed images with SHA-256 to detect exact duplicates and checked totals and diagonals in saved confusion matrices. Hashes detect byte-identical copies only. No perceptual similarity assessment or split-membership reconstruction was completed, so the audit cannot establish how many duplicates crossed the original training/validation boundary.

3.4 Measures and evaluation metrics

For N labeled images, accuracy is the number of correct predictions divided by N. For each class, precision is TP/(TP+FP), recall is TP/(TP+FN), and F1 is 2PR/(P+R), with zero-division handling as implemented by scikit-learn [12]. Macro F1 is the mean of class F1 values. The reports include these quantities and the full confusion matrix. The

training metric top-three accuracy is a separate ranking measure and is not substituted for top-one accuracy.

For organ detection, the saved report includes mAP at IoU 0.5 and mAP across IoU thresholds 0.5–0.95. Its bootstrap annotation condition must accompany those numbers. Planned additional measures include confidence calibration, unknown-input false acceptance, latency distributions, multi-image consistency and usability task completion. These measures have no recorded values in this thesis.

3.5 Data analysis

Saved report values were read directly and cross-checked against confusion-matrix totals. The active plant report records 14 correct predictions out of 15, seed/spice 59 out of 59, and flower_retrained 32 out of 35. The larger leaf report records 319 out of 359. Errors were examined by class rather than explained as proven causal effects of lighting or leaf shape.

Comparisons between artifacts are descriptive because class sets, supports and exact run conditions differ. The flower and flower_retrained reports have the same nominal support, but sample identity is not demonstrated by support alone. No significance test or claim of a fair head-to-head model comparison is made. The duplicate audit is used to qualify validity, not to calculate a corrected accuracy without original split records.

3.6 Ethics privacy security and responsible AI

The identification output is educational and must not authorize consumption or treatment. This is especially relevant to broad labels and visually similar species. The stored benefits records and their safety warnings are described as application content; the thesis does not repeat unverified therapeutic claims as established facts.

The backend re-encodes verification photographs as fresh JPEG images without copying EXIF metadata and stores a separate optional training-consent flag. These are implemented protections, but photographs can still contain identifying visual content. Retention rules, public-status authorization, reviewer accountability and removal procedures require operational definition. No ethics approval, participant consent or expert certification is invented.

Exact duplicate groups were observed in the current datasets. They should be grouped before future splitting, and related images of one specimen should also remain together. Image ownership and taxon labels must be checked before reuse; a dataset URL alone is insufficient evidence of permission.

3.7 Reproducibility and research artefacts

Reproduction requires the pinned branch, dataset manifests, ordered label files, compatible dependencies and an explicit training command. The classifier script fixes the split seed, but does not demonstrate comprehensive deterministic seeding across all training randomness. GPU characteristics, original run commands and wall-clock training durations are not preserved in the inspected reports.

Table 3.1 distinguishes script defaults from saved-model configurations. These defaults are not assumed to reproduce every historical training run. During this revision, all 18 tests in backend/tests passed at the documented commit on 10 October 2026. Section 5.4 records the command, environment and scope. This software check does not reproduce model training or archived recognition metrics; independent real-model and device evaluation remains outstanding.

Table 3.1 Training settings and artifact evidence

Setting	Current script default	Evidence limitation
Image size and batch	224 × 224; 32	Size corroborated by model input; original batch commands unavailable
Split	20 percent validation; seed 22168986	Stratified by class; specimen grouping absent
Backbone	EfficientNetV2B0	Active configs are B0; separate leaf artifact B1
Epoch limits	25 head; 10 fine tuning	Limits not verified for every original run
Learning rates	0.001; 0.00001	Scheduler may reduce rates
Head	256 ReLU units; dropout 0.45 and 0.35	Active flower first dropout 0.50; expanded leaf head 384
Loss	Sparse categorical cross entropy	Corroborated by active saved compile config

Setting	Current script default	Evidence limitation
Augmentation	Flip; rotation 0.12; zoom −0.10 to 0.22; translation 0.08; contrast 0.22	Code settings; no independent ablation
Fine tuning	Last 45 B0 or 65 B1 layers enabled	Current code; original command history unavailable
Export	Keras TFLite labels CSV report	Client uses backend Keras rather than on device TFLite

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The proposal calls for smartphone image capture, identification and related plant information [1]. The implementation expands input categories and provides multiple-image inference. Functional requirements derived from the actual component are summarized in Table 4.1. Their status denotes code presence, not a claim that each requirement has passed an end-to-end field test.

Table 4.1 Identification requirements and implementation status

ID	Requirement	Observed status
FR01	Capture or select up to five images	Present in identification page
FR02	Select leaf seed or flower classifier	Present; 6 10 and 8 active labels
FR03	Validate input and prepare image views	Present; model inference not rerun
FR04	Display confidence and alternatives	Present; warning threshold is uncalibrated
FR05	Display educational metadata and disclaimer	Present; source validation incomplete
FR06	Detect and classify a live plant organ	Present; bootstrap detector report
FR07	Submit and track expert review	Backend present; current identification UI absent
NFR01	Limit resources and handle unavailable service	25 MiB limit and 503 paths present
NFR02	Protect review images and credentials	EXIF removal and key gate present; operational policies missing
NFR03	Establish field accuracy and usability	No independent field or stakeholder result

Non-functional requirements concern response time, accessible interaction, model consistency, upload limits, privacy and clear uncertainty. The 60-second client timeout is an interaction setting, not a measured latency. The 25 MiB server request limit is a

resource constraint. Model/label dimension checks provide a concrete integrity safeguard, while field accuracy and scalable concurrent operation require separate measurements.

4.2 Component architecture within the complete solution

Figure 4.1 shows the identification component's verified boundary. The mobile client sends image bytes to the Flask service. The service validates the request, optionally checks the first image's organ category, prepares inference views and invokes the selected Keras classifier. It retrieves educational metadata and returns a JSON result. Live input adds YOLO-based organ localization before classification.

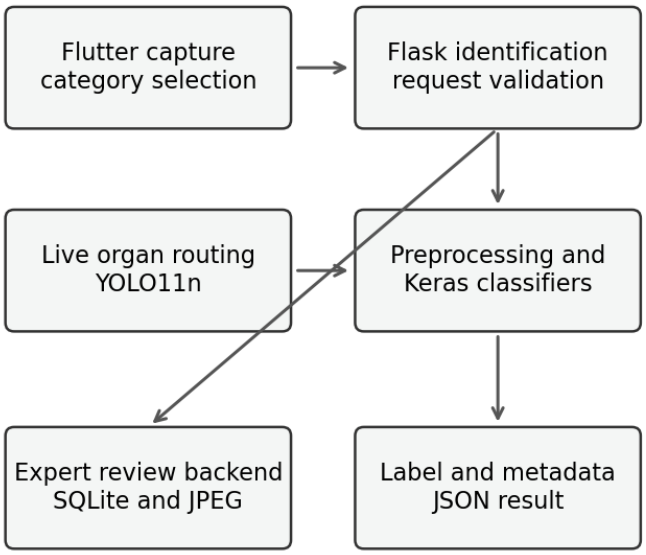


Figure 4.1 Identification component within the shared application

The backend's shared route module also hosts assistant, cultivation and quality routes. These are outside the individual identification boundary. Co-location creates deployment dependencies, but does not establish that plant labels are automatically transferred to every neighbouring component. The expert-review routes provide a separate backend workflow, which is not exposed by the currently inspected identification page.

4.3 Design alternatives and decisions

One alternative is a single classifier spanning all organs. The implemented design instead uses separate models selected by model_type. This makes category-specific class sets explicit, but introduces a selection risk if the user chooses the wrong category. Optional organ validation addresses a subset of that risk, because it checks only the first image and proceeds if the detector is unavailable or makes no qualifying detection.

A second choice concerns inference views. The service averages four center-crop scales and their horizontal mirrors, then averages across uploaded images. This can reduce dependence on one framing, but its benefit was not isolated in an ablation study. It also multiplies inference work and may mix evidence if the submitted photographs show different species.

A third choice is server-side Keras inference. TFLite files are exported, but the reviewed Flutter code sends requests to the backend and does not load them for on-device prediction. Offline classification is therefore a possible future design, not an implemented property. EfficientNetV2 and MobileNetV2 alternatives are discussed in Chapter 2 without claiming that a controlled architecture selection experiment was recorded.

4.4 Detailed design

4.4.1 Dataset Labels and Metadata Schema

Each classifier output index is interpreted through its ordered labels.json array. The model loader checks that the number of labels equals output_shape[-1]. Benefits records include common_name, scientific_name, traditional_uses, preparation_notes, safety_warning and medical_disclaimer. Supplemental entries update the base dictionary, so duplicate keys in the supplement take precedence.

Normalization removes non-alphanumeric characters and compares lowercase forms; a second attempt ignores parenthesized common-name suffixes. This is a convenient lookup mechanism, not a taxonomy service. Any unresolved label falls back to generic information with cautionary text. The fallback's prettified scientific_name is not a

validated scientific name and should be improved before a formal botanical catalogue is claimed.

4.4.2 Dataset Preparation and Training Pipeline

Dataset folders determine class labels and the script performs a stratified split. Online augmentation applies horizontal flips, rotation, zoom, translation and contrast variation. Because augmentation is part of the training model, it is not equivalent to independent new specimens. Data preparation should preserve specimen identity and image origin before the split; that provenance layer is currently missing.

The head uses global average pooling, batch normalization, dropout, a ReLU dense layer, another batch-normalization/dropout pair and softmax output. Keras documentation explains the EfficientNetV2 application interface and preprocessing option [13]. The script includes preprocessing inside EfficientNetV2 and accepts float image arrays rather than manually dividing all uploads by 255.

4.4.3 Prediction Pipeline and Confidence Warning

Figure 4.2 details the upload route. The preprocessing quality record reports original dimensions, brightness, sharpness, cropping and warnings. Four crop scales—1.0, 0.95, 0.90 and 0.80—each yield an original and mirrored view. All eight output vectors are averaged per image, then the image vectors are averaged equally.

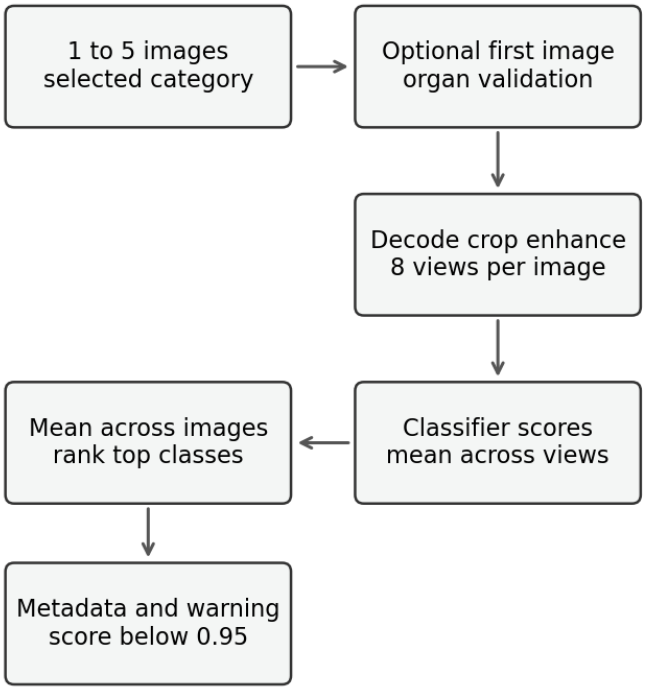


Figure 4.2 Upload preprocessing and prediction workflow

If p(i,v,c) is the class score for image i and view v, the final score for class c is the average across the eight views and the n submitted images. The predicted class is the maximum-scoring class. Up to five alternatives are returned. A score below 0.95 produces a warning but still returns a class. There is no explicit unknown-species output in this route, so confidence warning must not be described as open-set rejection.

4.4.4 Organ and Category Resolution

The organ detector recognizes leaf, seed and flower. It returns normalized bounding boxes and scores. For live identification, the backend selects the strongest detection, crops its region and maps leaf to the plant classifier while preserving seed and flower categories. If no organ is detected, identification is null rather than forcing a species result.

Category validation in the upload route runs at a 0.70 detection confidence setting. A confident mismatch returns a structured category_mismatch error and a suggested category. It does not test every uploaded image, and unavailability is handled by continuing to classification. These distinctions define its limits as a routing aid.

4.4.5 Result Presentation and Safety Wording

The result page displays the plant label, common and scientific names from metadata, confidence, alternative predictions, image-quality feedback, traditional-use text and warnings. It makes low-confidence feedback visible. The content is rendered from structured data rather than produced by a persona prompt.

The warning text encourages clearer photographs, but a better future formulation will distinguish a model score from verified correctness. Traditional-use records need traceable sources and qualified review. The result display should preserve this separation even when an identification score is high.

4.4.6 API and Client Design

The main upload contract uses repeated multipart fields named images and a model_type value of plant, seed or flower. The client adds validate_category=true. Table 4.2 lists identification-relevant endpoints and error conditions. Prediction and live scanning use different file field names and should not be treated as interchangeable.

Table 4.2 Identification relevant API contracts

Endpoint	Input or output	Conditions
GET /api/health	Dependency and artifact status	200 does not prove full inference
POST /api/predict	Repeated images; model_type; optional validate_category	400 invalid input; 503 unavailable model; warning below 0.95
POST /api/detect-organs	Single image; optional confidence	Normalized boxes; 400 or 503
POST /api/identify-live	Single image; optional confidence	Highest score box classified; no detections gives null identification
POST /api/verifications	images AI label AI confidence consent	201 pending request; 400 invalid input
GET /api/verifications/{id}	Public request status	404 unknown ID
GET or POST /api/expert/verifications/...	X-Expert-Key for queue images and review	401 unauthorized; protected key required

Endpoint	Input or output	Conditions
Request limit	Default total 25 MiB	413 JSON error

The Flutter service accepts an API_BASE_URL override, uses the browser host on web, 10.0.2.2 for the Android emulator and localhost for other default platforms. These settings support local development. Production URLs, HTTPS and device/network configuration require explicit deployment work.

4.5 Standards and professional practice

The design uses JSON responses, HTTP status codes and structured artifact checks. Upload failures are distinguished from unavailable models. A configurable request-size limit and CORS origins support deployment configuration. The present default permits all origins; production should restrict origins and protect expert credentials.

Reproducible professional practice also requires a dataset licence record, a taxon vocabulary, versioned model selection and signed-off evaluation records. Existing report filenames and artifact dates help trace work but do not replace a run manifest. No formal certification or compliance assessment is claimed.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The component uses Python, Flask, TensorFlow/Keras, Pillow, NumPy and scikit-learn, with Ultralytics for YOLO and Flutter/Dart for the client [2]. The dependency files pin Flask 3.0.3, Flask-CORS 4.0.0 and TensorFlow 2.21.0, while several other dependencies use version ranges. Saved active classifiers identify Keras 3.14.1 in their metadata. These are repository facts, not a verified reproduction environment.

The project guide recommends Python 3.11. Revision-stage verification used Python 3.12.14 on Linux x86_64 in a separate virtual environment inheriting the document runtime packages. The environment installed pytest 8.3.5, Flask 3.0.3, Flask-CORS 4.0.0, python-dotenv 1.0.1 and OpenCV headless 4.10.0.84. This was a software-test environment, not a complete isolated reproduction of all project dependencies. TensorFlow and Ultralytics were not installed, so classifier and detector inference were not reproduced.

5.2 Component implementation

5.2.1 Dataset Loading and Model Export

train_model.py sorts class directories, decodes recognized image files and assigns integer labels. It writes labels before fitting. The script uses class weights based on full-directory counts and supports EfficientNetV2B0, EfficientNetV2B1 and MobileNetV2. Exported Keras configurations confirm B0 backbones for all three active classifier paths and a B1 backbone for leaf_retrained.

The active plant artifact has six outputs, the active seed artifact ten and the active flower artifact eight. The separately exported 25-output B1 model has a 384-unit dense layer; the active B0 models have 256-unit dense layers. The flower artifact's first dropout is 0.50, compared with 0.45 in the active plant and seed artifacts. These values are read from saved configurations, not assumed from current defaults.

Training callbacks save the best val_accuracy checkpoint, stop after eight epochs without val_loss improvement and reduce learning rate on a plateau. A final model.save writes to the same filename after training, so a filename alone does not

prove it contains the globally best accuracy checkpoint. The CSV logger is reused between stages without append=True; the retained log cannot safely be interpreted as a full two-stage history.

5.2.2 Preprocessing Aggregation and Category Resolution

image_preprocessor.py corrects EXIF orientation and converts images to RGB. Images smaller than 32 pixels on either side are rejected. Mean grayscale brightness below 45 or above 220 creates lighting guidance, and an edge-based standard-deviation score below 12 creates a blur warning. These heuristic values are not validated measures of species-identification quality.

The crop heuristic samples at up to 320 by 320 pixels and estimates border background colour using a median. Cropping is skipped when border spread exceeds 38 or foreground coverage is outside 0.03–0.92. Otherwise it adds padding and avoids a crop retaining more than 90 percent of the original area. Autocontrast and small contrast/sharpness increases follow. The classifier service then resizes views to 224 by 224 with Lanczos interpolation.

Model loading is lazy and cached per category. It checks artifact existence and label dimensions. model_status checks dependency presence and files without fully loading every network; a healthy status is therefore not a successful inference smoke test. The actual model loader remains responsible for detecting incompatibilities when prediction is requested.

5.2.3 Plant Information and Human Review

The information dictionary is loaded alongside models and enriched with supplemental entries. It is a label lookup rather than a RAG retrieval step. Tests named benefits coverage check populated fields, but their names do not prove expert review. No individual record's therapeutic content is certified by the existence of that test.

expert_verification.py creates UUID request identifiers, stores pending records in SQLite and re-encodes uploaded images as JPEG. It stores the AI label and confidence, training consent, timestamps and eventual expert fields. Expert routes require an environment-defined key. Completed records are labeled Expert verified, while

pending ones retain AI identified. No completed botanical review study or automatic retraining loop is demonstrated.

5.2.4 API and Client

The identification page permits category selection, gallery input and camera input with quality 88. Gallery selection retains at most five photographs; camera capture appends and removes the oldest when necessary. Identification is initiated through the page action rather than automatically after every capture in the inspected current code. A request counter guards against stale responses updating the page.

PlantApi uses a 60-second timeout around the send operation and handles network, response-format and backend errors. That implementation should not be described as a measured 60-second end-to-end service guarantee. The current identification page contains no submission or status interface for expert review. The README's removed-frontend description and its 45-second timeout therefore cannot be substituted for current Flutter behaviour.

The live service calls identify-live with one image. The backend returns detection information and a compact identification object; the client parses organ and plant scores separately. Organ confidence and species confidence measure different outputs and should remain distinct in the display.

5.3 Integration with the other components

The identification component is integrated into the shared Flask backend and Flutter application. The current client sends repeated image fields and a selected model category to /api/predict, while live scanning sends one frame to /api/identify-live. The API returns a plant label, category, scores, image-quality feedback and educational information in structures parsed by the client. This agreement provides static evidence of integration, although device-based end-to-end testing was not independently reproduced.

The shared backend also contains routes for the herbal knowledge assistant, cultivation recommendations and quality assessment. These neighbouring components remain outside the individual identification scope. Co-location does not prove that a classifier label is automatically passed between every component. Any future handoff should

retain the model version, input category and confidence warning so that downstream information is not presented with greater certainty than the identification supports.

The expert-verification backend stores pending requests and later review outcomes, but the current identification page does not expose its submission or status workflow. The integration should therefore be described as an implemented backend interface with a remaining client task. No completed expert study, automatic retraining process or production deployment is inferred from the presence of these routes.

5.4 Verification and software testing

The repository contains API-contract, preprocessing, metadata-coverage and expert-verification tests, plus a separate Flutter interface test. On 10 October 2026, the backend/tests suite completed with 18 passed, zero failed and zero skipped in 0.32 seconds. Table 5.1 distinguishes the cases exercised by this run from additional proposed scenarios. Several API tests mock prediction or detection, so passing them does not establish recognition accuracy or successful loading of real models.

Table 5.1 Verification scenarios and evidence status

Case	Expected behaviour	Evidence status
T01 no images	400 and upload guidance	Passed; API test
T02 over five images	400 maximum count message	Not covered by the executed suite; additional case required
T03 missing model	503 service message	Additional real-model availability test required
T04 category mismatch	400 with suggested category	Passed; detector mocked
T05 invalid bytes	400 readable image error	Additional invalid-image case required
T06 plain border	Foreground crop around object	Passed; preprocessing test
T07 tiny image	Reject dimensions below 32	Passed; preprocessing test
T08 EXIF sanitization	Stored JPEG omits original EXIF	Passed; expert-verification storage test
T09 metadata coverage	Required information exists for model and dataset labels	Passed; two coverage tests; not botanical review

Case	Expected behaviour	Evidence status
T10 field recognition	Independent reviewed species labels	Proposed; no field result
T11 multi-image gain	Paired comparison against single-image input	Proposed; no ablation result
T12 unknown species	Measure false acceptance and warnings	Proposed; explicit unknown rejection absent
T13 report arithmetic	Matrix totals agree with recorded accuracy	Completed preparation-stage check; four reports
T14 duplicate scan	Count byte-identical images by SHA-256	Completed preparation-stage check; seven flower and one leaf groups

The revision-stage command was python -m pytest tests -q --junitxml=tests.xml, executed from backend at commit ae78e6d5abd60c521303ec66ad01a61dc3b4778c. The first attempt stopped during collection because OpenCV was missing; installing the declared OpenCV headless version allowed all 18 tests to pass. The JUnit result records ten API tests, two metadata-coverage tests, three expert-verification tests and three preprocessing tests. The run verifies the repository backend/tests suite only; Flutter tests, other project-component suites, real-model inference and device-level integration were not executed.

An end-to-end plan will test actual models through the API on a frozen independent set, then test mobile capture and error paths on nominated devices. The plan includes unsupported species, incorrect categories, corrupted files, five-image mixed-species requests, model unavailability and explicit medical-warning display. Executed results must be added with date, environment and logs before these cases are marked passed.

5.5 Deployment and operation

The backend exposes the API under /api and uses a 25 MiB default total request limit. It responds with JSON for 413, 404 and 405 conditions. Development execution uses port 5000. Production deployment is not verified by the presence of run.py or model artifacts.

Operation requires compatible ML dependencies, all three active models and labels, the YOLO weights for live detection, restricted CORS and a protected expert key.

Verification data need a persistence and backup policy. HTTPS, rate limits, per-user access to review status and a retention policy remain operational requirements. No measured concurrency, uptime or inference-time claim is included.

6 RESULTS AND VALIDATION

6.1 Research results

Table 6.1 reports the recorded validation metrics for the active classifier paths and the separate expanded leaf artifact. Their values were transcribed from evaluation_report.json and checked against matrix totals. They are archived results whose inference was not independently rerun during thesis preparation.

Table 6.1 Recorded classifier validation results

Artifact	Classes	N	Correct	Accuracy	Macro F1
Active leaf B0	6	15	14/15	93.33%	0.9429
Active seed B0	10	59	59/59	100.00%	1.0000
Active flower B0	8	35	32/35	91.43%	0.9224
Separate leaf B1	25	359	319/359	88.86%	0.8750

The active six-class leaf report records 93.33 percent accuracy on 15 validation images. Its only off-diagonal entry is one Andrographis paniculata image predicted as Azadirachta indica. The seed/spice report records 59 correct predictions from 59 validation images. That perfect result applies to the saved split only. The active flower_retrained report records 32 correct predictions from 35: Rose contributes one prediction to Crown Flower and one to Datura, while shankupushpam contributes one to Datura.

Figure 6.1 visualizes the active leaf matrix and Figure 6.2 the active flower matrix, both reconstructed from recorded counts. These are original plots of the component's reports, not copied diagrams or new predictions. The small supports mean that a single error changes the active leaf accuracy by 6.67 percentage points and flower accuracy by 2.86 points.

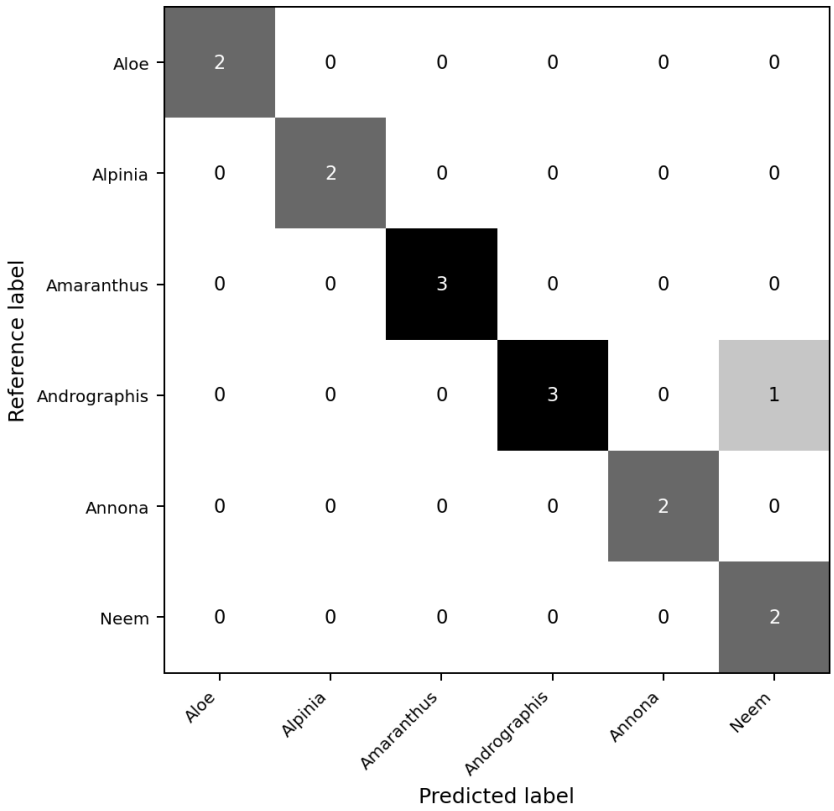


Figure 6.1 Recorded active leaf validation confusion matrix

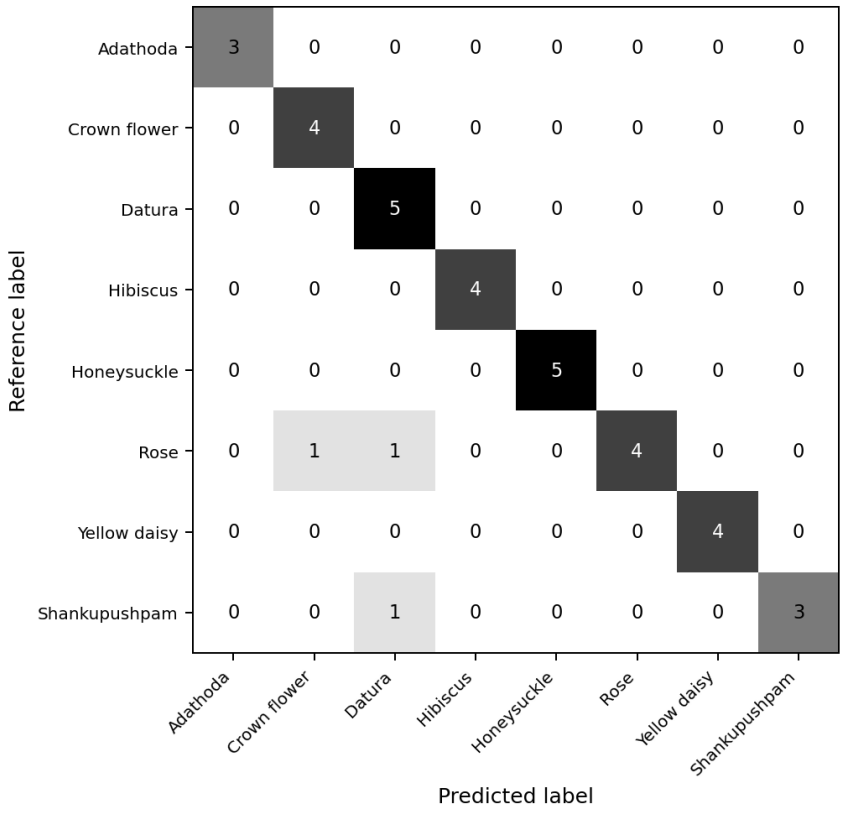


Figure 6.2 Recorded active flower validation confusion matrix

The separate 25-class leaf report records 88.86 percent accuracy on 359 validation images, with macro F1 of 0.8750. It is not the model selected by MODEL_ARTIFACTS['plant']. Its larger class set and support make the lower percentage unsuitable as evidence of regression or improvement relative to the six-class model. The earlier flower directory records 94.29 percent on 35 images but is likewise not selected for active flower inference.

6.2 Product and technical results

The pinned branch contains the image upload endpoint, category-specific models, JSON metadata lookup, live organ detection and review storage. This establishes implementation presence. Artifact metadata records May 10, 2026 for the active plant model and August 29, 2026 for the active seed and flower artifacts. These timestamps are serialization metadata, not submission dates or guaranteed training histories.

The saved YOLO report identifies YOLO11n, 591 training images, 147 validation images, image size 416 and CPU execution. It records precision 0.583, recall 0.687,

mAP50 0.6421 and mAP50–95 0.4074. Class mAP50 values are 0.4239 for leaf, 0.7696 for seed and 0.7327 for flower. The report labels its annotations as bootstrap full-frame boxes and recommends manually drawn boxes before production. Those conditions materially limit interpretation of localization performance. Ultralytics documentation establishes the detector's task interface [14], not these component-specific numbers.

The preparation-stage exact-hash audit found seven two-file duplicate groups in the current flower dataset and one two-file group in the leaf dataset; no exact duplicate group was found in seed/spice. These were current-tree observations. Whether matching files entered different original splits remains unverified. Matrix totals and diagonal sums agreed with the recorded accuracies. These arithmetic checks do not establish prediction provenance or absence of label error. Revision-stage software verification passed all 18 backend/tests cases; this result concerns software contracts and utilities, not independent recognition performance.

6.3 User or stakeholder validation

No completed usability study or documented botanical expert evaluation is available for this component. The presence of capture controls, category selection, warning messages and result fields demonstrates interface implementation, but does not establish user task success or correct interpretation. Stakeholder validation is therefore not reported as an achieved result, and no participant count, satisfaction score or expert agreement rate is claimed.

The proposed usability protocol covers image capture, category selection, interpretation of confidence warnings and locating the educational disclaimer. It records task completion, errors and user feedback after consent. A separate botanical evaluation requires qualified reviewers, independently identified specimens, a reference-label procedure and a documented method for resolving disagreements. These protocols are future work and are not presented as completed experiments.

6.4 Integration results

The Flutter service and Flask route agree on multipart images, category names and result fields. Live scanning uses a separate single-image contract. This provides static

contract evidence for integration. No recorded multi-device end-to-end run establishes successful operation across every target listed in the Flutter project.

The backend review interface is implemented, but the current identification page does not expose it. The README also says the frontend was removed, while the pinned tree contains a frontend. Code presence is treated as authoritative for this snapshot and the documentation discrepancy is recorded. There is no evidence that exported TFLite classifiers perform inference inside the mobile application.

6.5 Summary of findings

The component provides implemented classification, preprocessing and information-display functions, together with archived validation results for explicitly defined class sets. The evidence does not establish independent field accuracy, calibrated confidence, completed stakeholder validation or production readiness. The reported implementation should therefore be treated as a research prototype whose operational reliability remains to be measured.

The most consequential findings are the difference between trained and active leaf coverage, the mismatch between validation and API preprocessing, and the lack of specimen-level split records. These findings answer the research questions more usefully than reporting a single combined accuracy across unrelated classifiers.

7 DISCUSSION

7.1 Interpretation of findings

RQ1 is answered by the implemented separation of organ categories and classifier artifacts. RQ2 is partly answered by the saved metrics; it remains unanswered for independent field generalization. RQ3 is answered at the software level by structured warnings, metadata and result rendering. RQ4 has static interface evidence but lacks completed end-to-end and stakeholder evidence.

The seed result is encouraging within its split, but 59 images do not establish universal recognition. The active leaf support is particularly small. The flower errors identify labels needing attention but do not establish their cause. Additional images, verified specimen labels and direct API evaluation would provide a stronger basis for explaining failure mechanisms.

7.2 Comparison with literature baselines and existing solutions

Deep-Plant demonstrates learned representations for a different leaf dataset [3], while Leafsnap illustrates an identification-and-information application [8]. Mobile medicinal-herb work supplies local precedent [4]. The current component combines these broad ideas in a bounded project implementation rather than establishing a new category of software.

EfficientNetV2 provides a defensible architecture family [7] and MobileNetV2 remains an implemented training alternative [9]. However, the repository does not contain a matched identification benchmark proving which is best under a common field-image split. Differences in labels, image acquisition and evaluation prevent direct score comparisons with the literature. A future baseline study should use the same specimen-separated data, preprocessing, tuning budget and test conditions for each model.

7.3 Research contribution

The research contribution is the design and analysis of a category-specific identification workflow combining model selection, image preparation, uncertainty communication and educational metadata. The analysis establishes the distinction

between active and experimental class coverage and identifies limitations of the archived validation procedure. It does not claim a new recognition algorithm or a controlled improvement over existing systems.

The larger leaf artifact demonstrates expansion in training coverage, but its integration is incomplete in the selected prediction path. Identifying this boundary is part of an honest assessment. It prevents results for 25 classes from being attached to a six-class deployed service.

7.4 Product contribution

The product contribution is a mobile interaction backed by image processing, classifiers, alternative matches and educational records. It supports both prepared multi-image input and a live detection/classification route. The backend expert-review mechanism preserves a distinction between automatic and reviewed labels.

The present product value remains provisional. Unknown-species rejection, current-client expert submission and operation under production access controls need additional work. Exporting a TFLite file does not itself deliver offline use; a mobile interpreter and its validation would be needed.

7.5 Limitations and threats to validity

Internal validity is limited by reused validation data for tuning, missing specimen manifests, current exact duplicates and incomplete run history. Stratification preserves label proportions but cannot ensure independence between photographs of one plant. Exact hashes also miss resized or edited copies. The archived six-class audit and model support do not reconstruct a single fully traceable training dataset.

Construct validity is limited by treating softmax scores as confidence without calibration and by broad flower names that may not denote a single species. The crop-based API and resized-batch validation are different measurement procedures. Test names claiming reviewed benefits measure field presence rather than botanical or medical authority.

External validity remains unmeasured for Sri Lankan field scenes, new cameras, mixed plants and unsupported species. Background variation, class imbalance and visually

similar leaves are plausible risks, but only the listed matrix errors and current class counts were directly observed here. No conclusion about their causal contribution is presented as an experimental finding.

Deployment validity is limited by missing measured latency and device runs, permissive default CORS and an expert-review interface not present in the current identification page. Public review-status lookup is keyed by request identifier rather than demonstrated per-user authorization. These limitations require testing and design review before broad release.

7.6 Implications

For developers, the priority is to align active artifacts, labels and metadata under an explicit version manifest. For researchers, the priority is independent specimen-level testing rather than enlarging headline validation accuracy. For users, the priority is an understandable candidate label with uncertainty and safe interpretation.

For the overall project, a plant label can support other services only after its uncertainty and taxonomy are preserved. A high classifier score should not become an unquestioned input to medicinal or economic advice. Integration should carry the source model version, category and warning alongside the label.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

This thesis presents Intelligent Ayurvedic Herbal Plant Identification as the individual identification component of project R-26-IT-021. The implementation connects Flutter image capture to Flask endpoints, category-specific EfficientNetV2 classifiers, multi-view inference and educational JSON lookup. A YOLO detector supports live organ routing. Expert verification is implemented as a separate backend service; the current identification interface does not provide a complete user-facing review workflow.

Saved validation records provide evidence of classification performance on their respective splits, but they do not justify a universal accuracy or deployment claim. The six-class leaf model remains active despite a separate expanded artifact. Unverified provenance, possible duplicate-related leakage and a mismatch between validation and API preprocessing limit the strength of the results. The appropriate conclusion is that the component has documented implementation and provisional validation evidence, with independent assessment still required.

8.2 Achievement of objectives

Table 8.1 relates the objectives to evidence and remaining work. Implementation objectives are substantially represented in the branch, while field performance, information validation and stakeholder evidence remain incomplete. This assessment avoids treating a saved model file as achievement of every research objective.

Table 8.1 Objective achievement and remaining evidence

Objective	Supported achievement	Remaining work
O1	Active classifiers and expanded leaf artifact	Verify taxonomy; retain six-class baseline pending independent evaluation
O2	Quality heuristics eight view averaging warning	Ablation calibration unknown inputs
O3	Metadata lookup and readable result screen	Traceable sources expert content validation

Objective	Supported achievement	Remaining work
O4	Matching client and API contracts; live route	Device tests and current expert review UI
O5	Archived validation; dataset checks; 18 backend tests passed	Independent specimen test; real-model, device and user studies

8.3 Individual research and product contributions

The individual research scope is image-based Ayurvedic herbal plant identification and the presentation of its results. It excludes the neighbouring quality-assessment, economic-analysis and conversational-assistance components. The repository role assignment supports this scope. Shared route and client files are described as integration resources, without attributing every function or experiment in the shared project to the individual author.

The component-specific contribution is assessed through the documented identification functions and their recorded validation evidence. A verified contribution history is needed to attribute individual code changes, dataset preparation and training runs precisely. The thesis therefore distinguishes the assigned component from unverified claims of sole implementation or experimental ownership.

8.4 Future directions

First, establish an image provenance and taxon-label manifest, resolve permissions and group exact and near duplicates by specimen. Freeze independent training, validation and test memberships. Collect a new field-image test set with reviewed labels and image capture conditions before further model selection.

Second, evaluate the deployed API procedure directly. Compare one-image, multi-image and multi-crop modes with a shared test set; assess calibration and unknown-input rejection; and record latency and memory on nominated hardware. Retrain the organ detector with manually annotated boxes and evaluate localization on cluttered scenes.

Third, retain the six-class leaf configuration as the documented baseline until the separate 25-class model has passed endpoint-level evaluation. Any later model change must update the ordered labels, metadata and client coverage statements together. Add

a current-client expert-review flow if it remains a requirement, and test authorization, consent and retention. Introduce reviewed Sinhala and local names with traceable references for educational records.

Finally, complete the contribution record, original run manifests, dataset permissions and independent evaluation evidence. Final submission also requires the author and supervisor to complete the administrative and certification fields under the applicable institutional process. These requirements remain outside the results demonstrated by the present implementation and archived reports.

REFERENCES

[1] H. D. Sachintha, “Intelligent Ayurvedic Herbal Plant Identification and Economic Analysis in Sri Lanka,” Project Proposal Report R-26-IT-021, Sri Lanka Institute of Information Technology, Mar. 2026. Supplied PDF.

[2] Project R-26-IT-021, “Intelligent Ayurvedic Herbal Plant Identification and Economic Analysis in Sri Lanka,” GitHub repository, branch feature/ayrvedic_plant_recognition, commit ae78e6d5abd60c521303ec66ad01a61dc3b4778c, reviewed Oct. 10, 2026. https://github.com/dinee10/R26-IT-021/tree/feature/ayrvedic_plant_recognition

[3] S. H. Lee, C. S. Chan, P. Wilkin, and P. Remagnino, “Deep-Plant: Plant Identification with Convolutional Neural Networks,” arXiv:1506.08425, 2015. https://arxiv.org/abs/1506.08425

[4] L. P. D. S. Senevirathne et al., “Mobile-based Assistive Tool to Identify & Learn Medicinal Herbs,” in Proc. 2nd Int. Conf. Advancements in Computing, 2020, pp. 97–102, doi: 10.1109/ICAC51239.2020.9357247. Institutional publication record https://rda.sliit.lk/entities/publication/5bc6f7c5-b032-4544-9809-7c6c8719bd57

[5] A. Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in Int. Conf. Learning Representations, 2021. https://openreview.net/forum?id=YicbFdNTTy

[6] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On Calibration of Modern Neural Networks,” Proc. 34th Int. Conf. Machine Learning, PMLR, vol. 70, pp. 1321–1330, 2017. https://proceedings.mlr.press/v70/guo17a.html

[7] M. Tan and Q. Le, “EfficientNetV2: Smaller Models and Faster Training,” Proc. 38th Int. Conf. Machine Learning, PMLR, vol. 139, pp. 10096–10106, 2021. https://proceedings.mlr.press/v139/tan21a.html

[8] N. Kumar et al., “Leafsnap: A Computer Vision System for Automatic Plant Species Identification,” in Computer Vision ECCV 2012, LNCS, vol. 7573, pp. 502–516, 2012, doi: 10.1007/978-3-642-33709-3_36.

[9] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” Proc. IEEE Conf. Computer Vision and Pattern Recognition, pp. 4510–4520, 2018, doi: 10.1109/CVPR.2018.00474. https://openaccess.thecvf.com/content_cvpr_2018/html/Sandler_MobileNetV2_Inverted_Residuals_CVPR_2018_paper.html

[10] Pl@ntNet, “Pl@ntNet a citizen science project,” official documentation, accessed Oct. 10, 2026. https://docs.plantnet.org/en/understand/plantnet-a-citizen-science-project/

[11] AI4A Lab, “Herb (Plant) Classification Dataset,” Kaggle dataset landing page cited by PROJECT_GUIDE.md, accessed Oct. 10, 2026. https://www.kaggle.com/datasets/ai4a-lab/herb-plant-classification-dataset. Detailed provenance and licence not verified in this review.

[12] scikit-learn developers, “classification_report,” official documentation, accessed Oct. 10, 2026. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html

[13] Keras developers, “EfficientNetV2 B0 to B3 and S M L,” official API documentation, accessed Oct. 10, 2026. https://keras.io/api/applications/efficientnet_v2/

[14] Ultralytics, “Ultralytics YOLO11,” official documentation, accessed Oct. 10, 2026. https://docs.ultralytics.com/models/yolo11/

GLOSSARY

Active artifact A model path selected by the current prediction service.

Candidate identification The model's best available label; it is not botanical certification.

Closed set classifier A classifier restricted to known labels without a dedicated unknown class.

Calibration Agreement between reported scores and observed correctness frequencies.

Validation set Data used during model development; not an untouched independent test set.

Specimen grouping Keeping photographs of the same biological specimen in one split.

Bootstrap box A generated approximate bounding box rather than a manually verified object boundary.

Recorded result An output preserved in the repository, distinct from independently rerun inference.

Exact duplicate Two image files with identical bytes according to SHA256.

Traditional use record Stored educational text that does not by itself establish therapeutic evidence.

APPENDICES

Appendix A Research instruments or extended results

A1 Supported labels and current class distribution

Table A.1 lists current image folders in sorted order. Only six labels are active for leaf inference; the remaining leaf labels belong to the expanded dataset and separate artifact. Counts are current files, not independent specimens. Seed and flower folders match their active label sets. Names are preserved as stored, without silently correcting taxonomy.

Table A.1 Current image class distributions

Category	Stored class label	Files	Active
Leaf	Allium cepa (Onion)	88	No
Leaf	Aloe barbadensis miller (Aloevera)	71	Yes
Leaf	Alpinia Galanga (Rasna)	50	Yes
Leaf	Amaranthus Viridis (Arive-Dantu)	122	Yes
Leaf	Andrographis paniculata (Nelavembu)	68	Yes
Leaf	Annona squamosa (Seethapala)	59	Yes
Leaf	Artocarpus heterophyllus (Jackfruit)	107	No
Leaf	Azadirachta indica (Neem)	70	Yes
Leaf	Bacopa monnieri (Bhrami)	82	No
Leaf	Bambusa vulgaris (Bamboo)	48	No
Leaf	Basella alba (Malabar Spinach)	161	No

Category	Stored class label	Files	Active
Leaf	Brassica Juncea (Indian Mustard)	34	No
Leaf	Brassica oleracea (Kohlrabi)	25	No
Leaf	Calotropis gigantea (Crown flower_Ekka)	35	No
Leaf	Capsicum annuum (Chilly)	24	No
Leaf	Cardiospermum halicacabum (Balloon Vine)	38	No
Leaf	Carica papaya (Papaya)	112	No
Leaf	Carissa Carandas (Karanda)	74	No
Leaf	Catharanthus roseus (Catharanthus)	71	No
Leaf	Chamaecostus cuspidatus (Insulin)	68	No
Leaf	Cinnamomum camphora (Camphor)	38	No
Leaf	Citrus limon (Lemon)	146	No
Leaf	Citrus medica (herelikai)	62	No
Leaf	Coffea arabica (Coffee)	42	No
Leaf	Coleus amboinicus (Mexican mint_Doddpathre)	98	No
Seed spice	Belleric Myrobalan	30	Yes
Seed spice	Black Pepper	30	Yes
Seed spice	Chebulic Myrobalan	30	Yes
Seed spice	Clove	30	Yes
Seed spice	Flax Seed	30	Yes

Category	Stored class label	Files	Active
Seed spice	Garlic	29	Yes
Seed spice	Gooseberry	25	Yes
Seed spice	Psoralea Fruit	30	Yes
Seed spice	Star Anise	30	Yes
Seed spice	Turmeric	30	Yes
Flower	Adathoda	16	Yes
Flower	Crown Flower	21	Yes
Flower	Datura	25	Yes
Flower	Hibiscus	20	Yes
Flower	Honey Suckle	22	Yes
Flower	Rose	31	Yes
Flower	Yellow Daisy	18	Yes
Flower	shankupushpam	18	Yes

A2 Implementation evidence register

Table A.2 provides repository-relative evidence paths at the pinned commit. Shared quality and RAG results are excluded from identification metrics.

Table A.2 Implementation evidence register

Path	Evidence
README.md	Individual role assignment; historical frontend handoff is partly stale
PROJECT_GUIDE.md	Setup rationale and dataset reference; not proof of licence
backend/app/services/predictor.py	Active paths scores crops metadata and 0.95 warning
backend/app/services/image_preprocessor.py	Orientation dimensions border cropping and quality heuristics
backend/app/services/organ_detector.py	YOLO path three organ classes and normalized boxes

Path	Evidence
backend/app/routes/api.py	Upload live routing category validation and expert routes
backend/app/main.py	Request limit CORS blueprint and JSON errors
backend/app/services/expert_verification.py	SQLite schema JPEG sanitization consent and review status
backend/ml/scripts/train_model.py	Split augmentation two stages export and validation routine
backend/ml/exports/{root,seed,flower_retrained}/	Active classifiers labels reports and logs
backend/ml/exports/leaf_retrained/	Separate 25 class B1 artifact and report
backend/ml/metadata/	Audits and base benefits; historical counts need reconciliation
backend/app/data/benefits_supplement.json	Supplemental metadata lookup entries
backend/prepare_yolo_dataset.py	Full frame bootstrap annotations
backend/yolo_training_report.json	Recorded organ metrics and annotation caveat
backend/tests/	18 tests passed during revision on 10 October 2026; ML inference not exercised
frontend/lib/screens/identify_plant_page.dart	Current category controls capture and manual request
frontend/lib/services/plant_api.dart	Multipart contract timeout and base URLs
frontend/lib/models/plant_prediction.dart	Result schema and fallback safety text
frontend/test/identify_plant_page_test.dart	UI test source; not executed here

A3 Independent evaluation instrument

Proposed procedure. Create a frozen image manifest with file hash, specimen ID, verified taxon, organ category, source permission and capture conditions. Separate specimens and related images across training, validation and test. Resolve exact and near duplicates before allocating splits. Record exclusions and preserve the final test set unchanged throughout tuning.

Run the active models through /api/predict with the same endpoint preprocessing used by the client. Store image IDs reference labels returned scores warnings ranked

predictions model hashes request category and timing. Report per class support precision recall F1 and confusion matrices. Compare single image and multi image inputs using paired specimens. Evaluate the organ detector separately using manually annotated boxes and then measure end to end species outcomes.

Add an unsupported input collection containing out-of-class plants, non-plant images, mixed species and deliberately wrong categories. Measure false acceptance rather than assuming a confidence warning rejects the input. Analyze calibration and select any rejection threshold on validation data only. Freeze that threshold before independent testing. Record hardware, software, network, warm-up procedure and run count for latency measurements.

For usability, define capture, category-selection, warning-interpretation and disclaimer-location tasks. Recruit with approved consent and report participant characteristics only after collection. For expert verification specify reviewer qualifications, reference identification procedure, disagreement handling and consented data reuse. These instruments are plans; no sample size or outcome is invented.

A4 Outstanding evidence and validation requirements

Authorship and certification. The supplied proposal identifies the student as H D Sachintha, IT22168986. An expanded legal name, the actual submission date, signed declarations and supervisor certification are not established by the technical evidence. Signature fields remain unsigned.

Institutional requirements. The existing declaration layout is retained. The applicable declaration wording and permitted scope of AI assistance have not been verified; Appendix B records the assistance actually used without asserting institutional approval.

Contribution history. The repository assigns the identification component to the student, but no approved file-level contribution record establishes sole ownership of shared code, datasets or experiments.

Dataset provenance. Per-image source, licence, acquisition date, specimen identity, verified taxon and reviewer records are absent. Exact and near duplicates must be resolved before creating specimen-separated, frozen partitions.

Historical data linkage. The 76-image historical audit and the active 15-image validation report lack a common split manifest. Their exact relationship is not established.

Run reproducibility. Original training commands, hardware, complete stage histories and model-to-report manifests are incomplete. A clean reproduction requires dated logs, software versions and hashes of the evaluated artifacts.

Independent evaluation. Field accuracy, calibration, unsupported-input false acceptance, pipeline ablation, latency and device results are not available. Appendix A3 defines the proposed evaluation scope.

Software and stakeholder verification. Revision-stage execution passed all 18 tests in backend/tests on 10 October 2026. This does not replace real-model API smoke tests, device-level integration, consented usability research or qualified botanical evaluation. Those activities remain outstanding.

Taxonomy and educational content. Stored class labels are retained for reproducibility, including their original spelling and capitalization. They are not a verified taxonomic catalogue. Broad flower labels require specimen-level identification, and educational metadata requires traceable sources and qualified review.

Appendix B Academic integrity and AI use disclosure

AI tools assisted in preparing the thesis for the Intelligent Ayurvedic Herbal Plant Identification component by supporting source inspection, content organization, drafting, language refinement, table and figure preparation, and formatting. The component focuses on identifying Ayurvedic herbal plants from images of leaves, flowers, and seeds or spices using deep learning models within a mobile application and backend service. The author remains responsible for verifying the technical descriptions, acknowledging all sources, accurately reporting individual contributions, and ensuring that the final submission follows the institution’s academic integrity and

AI use requirements. The reference thesis was used only to guide the structure, chapter order, and formatting; its component-specific content, results, diagrams, and personal information were not reused.

The technical discussion and archived recognition results are based on the component's repository code, model artifacts and evaluation outputs at the documented commit. AI-assisted revision also executed the 18-test backend/tests suite and recorded its passing result separately from the student's original experiments. Model training, real-model inference and participant studies were not repeated. Dataset counts, duplicate checks and confusion-matrix arithmetic are preparation-stage verification activities. No experimental accuracy, participant feedback, expert approval or signature was invented. Identification results do not establish medicinal effectiveness or suitability for consumption or treatment.

This statement records the AI assistance used in preparing the thesis. It does not assert that the institution has approved that assistance or prescribed this wording. The author must verify the applicable policy and complete the institution's required declaration before final submission.